

Request Handling

In Flask, **request handling** is the entire lifecycle of what happens when a client (like a web browser) sends an HTTP request to your application, how Flask processes it, and how it sends back a response.

Flask provides a global request object that contains all the incoming data sent by the browser.

1. The Request Lifecycle

Whenever a user interacts with your Flask application, the request goes through a specific pipeline:

- 1. The Request Arrives:** The client sends an HTTP request (e.g., hitting a URL or submitting a form).
- 2. Pre-processing (before_request):** Flask runs any functions scheduled to execute *before* reaching the route (e.g., checking if a user is logged in).
- 3. Route Matching & View Execution:** Flask matches the URL to a route and runs its corresponding view function. The request object is populated with data here.
- 4. Post-processing (after_request):** Flask runs functions scheduled to execute *after* the view function finishes (e.g., logging the request, cleaning up resources, etc.).

2. Extracting Data with the request Object

To handle incoming data, you must import the request object from the flask library. Depending on how the data is sent, you will access different attributes of this object.

A. URL Query Parameters (`request.args`)

Query parameters are key-value pairs appended to the end of a URL after a question mark (e.g., `/search?q=flask&page=2`).

To execute this code, you need to add the standard block at the bottom that tells Python to launch Flask's built-in development server.

Here is the complete code and the steps to run and test it.

1. The Complete Code

Create a file named app.py and paste this code inside:
Python

Python

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/search')
def search():
    # Use .get() to prevent crashing if the parameter is missing
    query = request.args.get('q', default='')
    page = request.args.get('page', default=1, type=int)

    return f"Searching for '{query}' on page {page}"

# This block is required to start the server when running the file directly
if __name__ == '__main__':
    app.run(debug=True)
```

← ↻ ⓘ 127.0.0.1:5000/search?q=flask&page=3

Searching for 'flask' on page 3

← ↻ ⓘ 127.0.0.1:5000/search?q=python

Searching for 'python' on page 1

← ↻ ⓘ 127.0.0.1:5000/search

Searching for " on page 1

How to Test and Provide URL Parameters

Because this code uses `request.args`, it is expecting data to be appended to the URL as **Query Parameters**. You can test this entirely inside your standard web browser.

Open your browser and try entering these different URLs to see how Flask extracts the data:

- **Test 1 (Passing both parameters):** Go to: `http://127.0.0.1:5000/search?q=flask&page=3`
- *Output:* Searching for 'flask' on page 3
- **Test 2 (Passing only the search query):** Go to: `http://127.0.0.1:5000/search?q=python`
- *Output:* Searching for 'python' on page 1 (*The page defaults to 1 because of your `default=1` fallback code*).
- **Test 3 (Visiting with zero parameters):** Go to: `http://127.0.0.1:5000/search`
- *Output:* Searching for " on page 1 (*The app doesn't crash; it safely falls back to your defaults*).

